Homework 10 — STAT 432 (Fall 2026)

Name: Owen Park **NetID:** owenp3

Everything is in R with the `RLT` package (version 6.1.1 from the author's r-universe), because the Python `rlt` package is not on PyPI and its GitHub page still lists the Python API as planned. Every forest is an ordinary random forest with `reinforcement = FALSE`. Question 1 uses the single seed **43210** in `set.seed`, and Question 2 calls `set.seed(43210)` again before its first forest. Each forest gets its own `RLT` seed drawn from R's generator, so the whole report reproduces from these two calls.

Question 1: Bias-variance trade-off simulation

The 200 test points are drawn once from the five-dimensional normal with unit variances and all correlations 0.3, and they never change. In each of the 100 repetitions, I draw 200 new training observations and fit every setting on that same training set, with 500 trees that each draw 200 observations with replacement (`resample.prob = 1`). For a setting, the squared bias at a test point is the squared gap between its average prediction over the 100 repetitions and $f(\mathbf{x}) = x_1 + x_3 + x_5$. The variance is the spread of the 100 predictions around that average, and both are then averaged over the 200 points. The third block of settings (100 observations per tree) is the extra simulation for the last part of 1b.

In [1]:

```
suppressPackageStartupMessages({library(RLT); library(MASS)})
options(repr.plot.res = 120, width = 120)
set.seed(43210)                                  # the one seed for all of Question 1

p <- 5
Sigma <- matrix(0.3, p, p); diag(Sigma) <- 1       # unit variances, all correlations 0.3
f_true <- function(x) x[, 1] + x[, 3] + x[, 5]
x_test <- mvrnorm(200, rep(0, p), Sigma)          # drawn once and kept fixed
f_test <- f_true(x_test)

settings <- rbind(data.frame(study = "node size", mtry = 3, nmin = c(1, 5, 10, 20, 30), prob = 1),
                  data.frame(study = "mtry", mtry = 1:5, nmin = 10, prob = 1),
                  data.frame(study = "node size, 100 per tree", mtry = 3, nmin = c(1, 5, 10, 20, 30), prob = 0.5))
n_rep <- 100
pred <- array(NA_real_, c(nrow(settings), n_rep, nrow(x_test)))

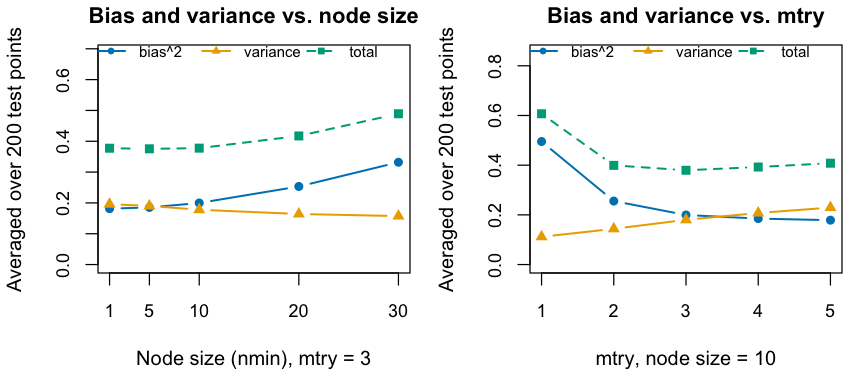
for (r in 1:n_rep) {
  x <- mvrnorm(200, rep(0, p), Sigma)
  y <- f_true(x) + rnorm(200)                    # one training set, shared by every setting
  for (s in seq_len(nrow(settings))) {
    fit <- RLT(x, y, model = "regression", ntrees = 500,
               mtry = settings$mtry[s], nmin = settings$nmin[s],
               resample.replace = TRUE, resample.prob = settings$prob[s],
               reinforcement = FALSE, ncores = 1, seed = sample.int(1e8, 1))
    pred[s, r, ] <- predict(fit, x_test)$Prediction
  }
}

# per test point: bias^2 = (mean prediction - f(x))^2, variance = variance over the 100 repetitions
settings$bias2    <- apply(pred, 1, function(P) mean((colMeans(P) - f_test)^2))
settings$variance <- apply(pred, 1, function(P) mean(apply(P, 2, var)))
settings$total    <- settings$bias2 + settings$variance
print(transform(settings, bias2 = round(bias2, 3), variance = round(variance, 3), total = round(total, 3)), row.nam
```

```
                study mtry nmin prob bias2 variance total
            node size    3    1  1.0 0.181    0.196 0.377
            node size    3    5  1.0 0.185    0.190 0.375
            node size    3   10  1.0 0.200    0.178 0.378
            node size    3   20  1.0 0.253    0.164 0.417
            node size    3   30  1.0 0.332    0.157 0.489
                 mtry    1   10  1.0 0.495    0.112 0.607
                 mtry    2   10  1.0 0.256    0.144 0.399
                 mtry    3   10  1.0 0.199    0.180 0.380
                 mtry    4   10  1.0 0.185    0.208 0.393
                 mtry    5   10  1.0 0.179    0.229 0.408
 node size, 100 per tree    3    1  0.5 0.244    0.134 0.378
 node size, 100 per tree    3    5  0.5 0.252    0.129 0.382
 node size, 100 per tree    3   10  0.5 0.288    0.121 0.409
 node size, 100 per tree    3   20  0.5 0.421    0.116 0.537
 node size, 100 per tree    3   30  0.5 0.610    0.118 0.727
```

In [2]:

```
okabe <- c(bias = "#0072B2", var = "#E69F00", total = "#009E73")
bv_plot <- function(d, xvar, xlab, main) {
  par(mar = c(4, 4, 2.5, 1), cex.main = 1.1, cex.lab = 1, cex.axis = 0.9)
  plot(d[[xvar]], d$total, type = "n", ylim = c(0, max(d$total) * 1.4), xaxt = "n",
       xlab = xlab, ylab = "Averaged over 200 test points", main = main)
  axis(1, at = d[[xvar]])
  lines(d[[xvar]], d$bias2,    type = "b", pch = 16, lwd = 1.6, col = okabe["bias"])
  lines(d[[xvar]], d$variance, type = "b", pch = 17, lwd = 1.6, col = okabe["var"])
  lines(d[[xvar]], d$total,    type = "b", pch = 15, lwd = 1.6, col = okabe["total"], lty = 2)
  legend("top", c("bias^2", "variance", "total"), col = okabe, horiz = TRUE,
         pch = c(16, 17, 15), lty = c(1, 1, 2), lwd = 1.6, bty = "n", cex = 0.75)
}
options(repr.plot.width = 7.2, repr.plot.height = 3.2)
par(mfrow = c(1, 2))
bv_plot(subset(settings, study == "node size"), "nmin", "Node size (nmin), mtry = 3", "Bias and variance vs. node s
bv_plot(subset(settings, study == "mtry"), "mtry", "mtry, node size = 10", "Bias and variance vs. mtry")
```



1b. Discussion

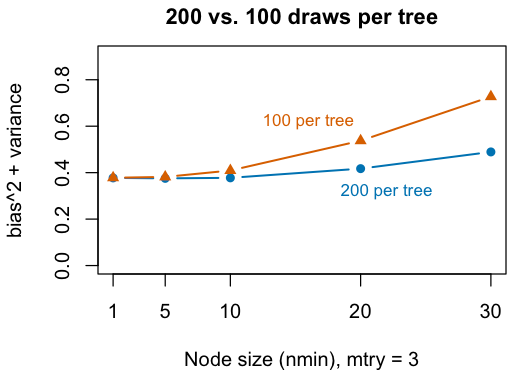
Large node size. Node size 30 adds bias and removes only a little variance. Going from node size 1 to 30, the squared bias rises from 0.181 to 0.332 while the variance falls from 0.196 to 0.157, so the total goes from 0.377 to 0.489. The true function is a plane that keeps rising in three directions, and a terminal node holding 30 of the 200 draws averages over a wide region of it, which flattens the fit and pulls the predictions toward the middle. Up to node size 10 the two effects roughly cancel (0.377, 0.375, 0.378), so in this example small nodes are the safe choice and anything past 10 costs accuracy.

Large mtry. A larger `mtry` lets each split find one of the three signal variables more often, which is why the squared bias falls from 0.495 at `mtry = 1` to 0.179 at `mtry = 5`. The drawback is that the trees become more alike, since they keep choosing the same strong splits, so averaging them cancels less noise and the variance climbs from 0.112 to 0.229. With `mtry = 1`, two of every five candidate draws are noise variables, and the bias dominates. The total is lowest at `mtry = 3` (0.380) and nearly flat from 2 to 5, so the middle values balance the two.

100 observations per tree. I expected the preferred node size to move toward smaller values, and the extra simulation agrees. With 100 draws per tree, a node of 30 covers almost a third of each tree's sample, so the same node size means a much coarser partition. The

bias at node size 30 jumps to 0.610, against 0.332 with 200 draws. The smaller samples also make the trees less alike, which already lowers the variance (0.134 at node size 1 against 0.196), so there is less variance left for large nodes to remove.

In [3]:
```
options(repr.plot.width = 4.4, repr.plot.height = 3.2)
par(mfrow = c(1, 1))
both <- subset(settings, grepl("node size", study))
par(mar = c(4, 4, 2.5, 1), cex.main = 1.1)
plot(c(1, 30), c(0, max(both$total) * 1.25), type = "n", xaxt = "n", xlab = "Node size (nmin), mtry = 3",
     ylab = "bias^2 + variance", main = "200 vs. 100 draws per tree")
axis(1, at = c(1, 5, 10, 20, 30))
for (k in 1:2) {
  d <- subset(both, prob == c(1, 0.5)[k])
  lines(d$nmin, d$total, type = "b", pch = c(16, 17)[k], lwd = 1.6, col = c("#0072B2", "#D55E00")[k])
}
text(c(16, 22), c(0.62, 0.32), c("100 per tree", "200 per tree"), col = c("#D55E00", "#0072B2"), cex = 0.85)   # di
```



With 100 draws per tree, the total is lowest at node size 1 (0.378) and rises at every step after that, while with 200 draws it is flat through node size 10. The minimum moves to the smallest node size and the penalty for large nodes gets steeper, which matches the reasoning above. The difference between node sizes 1 and 5 is small in both cases, so I read this as a shift toward small nodes rather than a sharp new optimum.

Question 2: Tuning random forests for MNIST

The first 1,000 rows give 327 training images of 1, 6 and 7 and the last 1,000 give 317 test images. The PCA is fit on the training pixels only, centered but not scaled, and the test images are centered with the training means and multiplied by the training loadings. The first 20 components keep 77.1% of the training pixel variance. Every forest below has 1,000 trees and draws a bootstrap sample of 327 with replacement for each tree. The OOB error is the share of training images whose out-of-bag vote is wrong.

In [4]:
```
mnist <- read.csv(gzfile("data/mnist-first2000.csv.gz"))
keep <- c(1, 6, 7)
train <- mnist[1:1000, ];    train <- train[train$Digit %in% keep, ]
test  <- mnist[1001:2000, ]; test  <- test[test$Digit %in% keep, ]
y_train <- factor(train$Digit); y_test <- factor(test$Digit, levels = levels(y_train))

pca <- prcomp(as.matrix(train[, -1]), center = TRUE, scale. = FALSE)   # fit on training pixels only
z_train <- pca$x[, 1:20]
z_test  <- scale(as.matrix(test[, -1]), center = pca$center, scale = FALSE) %*% pca$rotation[, 1:20]
cat("training:", nrow(z_train), "  test:", nrow(z_test), "\n")
print(rbind(train = table(y_train), test = table(y_test)))
cat("variance explained by 20 PCs:", round(sum(pca$sdev[1:20]^2) / sum(pca$sdev^2), 3), "\n")
```

```
training: 327   test: 317
        1   6   7
train 116  94 117
test  104 106 107
variance explained by 20 PCs: 0.771
```

2a. Default forest

In [5]:
```
set.seed(43210)
rf_fit <- function(mtry, nmin, importance = FALSE)
```

```
  RLT(z_train, y_train, model = "classification", ntrees = 1000, mtry = mtry, nmin = nmin,
      resample.replace = TRUE, resample.prob = 1, reinforcement = FALSE,
      importance = importance, ncores = 1, seed = sample.int(1e8, 1))
oob_error <- function(fit) mean(fit$Prediction != y_train)     # fit$Prediction holds the OOB votes

default_fit <- RLT(z_train, y_train, model = "classification", ntrees = 1000,
                   resample.replace = TRUE, resample.prob = 1, ncores = 1, seed = sample.int(1e8, 1))
cat("default mtry =", default_fit$parameters$mtry, " nmin =", default_fit$parameters$nmin,
    " OOB misclassification =", round(oob_error(default_fit), 4), "\n")
```

```
default mtry = 10  nmin = 5  OOB misclassification = 0.0428
```

With the default settings (`mtry` = 10, half of the 20 components, and node size 5) the OOB misclassification rate is 4.28%, which is 14 of the 327 training images.

2b. Tuning grid

The grid crosses eight `mtry` values from 1 to 20 with five node sizes from 1 to 20, which is 40 forests. I pick the setting with the lowest OOB error, and when settings tie I take the larger node size and then the smaller `mtry`, since those give the simpler forest.

In [6]:
```
grid <- expand.grid(mtry = c(1, 2, 3, 5, 7, 10, 15, 20), nmin = c(1, 2, 5, 10, 20))
grid$oob <- mapply(function(m, n) oob_error(rf_fit(m, n)), grid$mtry, grid$nmin)
oob_table <- xtabs(oob ~ mtry + nmin, grid)
print(round(oob_table, 4))
tied <- grid[grid$oob == min(grid$oob), ]
best <- tied[order(-tied$nmin, tied$mtry)[1], ]   # ties: larger node size, then smaller mtry
cat("\nselected: mtry =", best$mtry, " nmin =", best$nmin, " OOB error =", round(best$oob, 4),
    " (", sum(grid$oob == min(grid$oob)), "setting(s) tie at the minimum)\n")
```

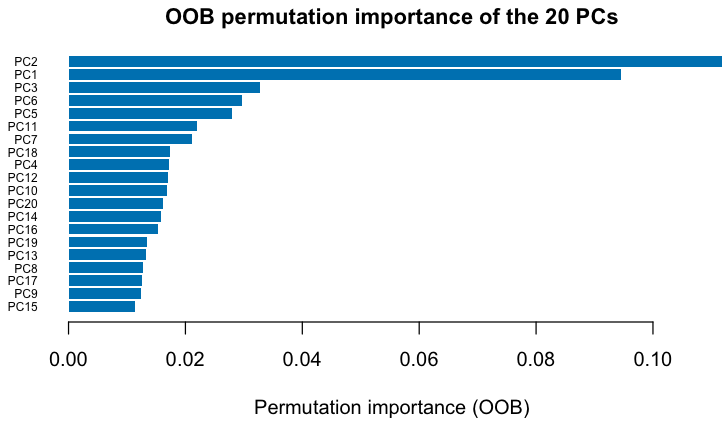
```
    nmin
mtry      1      2      5     10     20
  1  0.0214 0.0214 0.0245 0.0214 0.0275
  2  0.0336 0.0367 0.0306 0.0306 0.0428
  3  0.0367 0.0336 0.0367 0.0367 0.0367
  5  0.0367 0.0367 0.0367 0.0367 0.0398
  7  0.0398 0.0398 0.0398 0.0398 0.0367
  10 0.0398 0.0398 0.0459 0.0459 0.0428
  15 0.0489 0.0489 0.0489 0.0489 0.0520
  20 0.0489 0.0489 0.0520 0.0520 0.0520

selected: mtry = 1  nmin = 10  OOB error = 0.0214  ( 3 setting(s) tie at the minimum)
```

In [7]:
```
final_fit <- rf_fit(best$mtry, best$nmin, importance = TRUE)
cat("selected forest refit with importance: OOB error =", round(oob_error(final_fit), 4), "\n")
imp <- sort(setNames(final_fit$VarImp[, 1], colnames(z_train)), decreasing = TRUE)
options(repr.plot.width = 6.5, repr.plot.height = 3.6)
par(mar = c(4, 4, 2.5, 1), cex.main = 1.1)
barplot(rev(imp), horiz = TRUE, las = 1, col = "#0072B2", border = NA, cex.names = 0.6,
        xlab = "Permutation importance (OOB)", main = "OOB permutation importance of the 20 PCs")
round(imp[1:6], 4)
```

```
selected forest refit with importance: OOB error = 0.0245
```

PC2: 0.1118 **PC1:** 0.0946 **PC3:** 0.0327 **PC6:** 0.0298 **PC5:** 0.028 **PC11:** 0.022



The OOB error depends on `mtry` far more than on node size. It is lowest at `mtry = 1`, where 7 of 327 images (2.14%) are wrong at node sizes 1, 2 and 10. It then rises steadily to about 5% at `mtry` 15 and 20, while within a row the node size moves the error by at most four images. With my tie rule, the selected forest is **`mtry = 1`, node size 10**, with OOB error 2.14%, half the default's 4.28%. Small `mtry` works here because the components are

uncorrelated by construction and several of them carry signal. Forcing each split onto a random component keeps the trees different from each other, and the averaging gains more from that than it loses from weaker single splits. The gap between the best and the default is 7 images, but the next settings are only one or two images worse, and refitting the selected setting with a new seed for the importance run gives 2.45% (8 images). The ranking near the top is therefore within OOB noise. The importance agrees with what the digits look like. PC2 (0.112) and PC1 (0.095) dominate, with PC3, PC6 and PC5 next at about 0.03. The components past PC11 contribute almost nothing. The first two components already separate the three digits, and the rest only fix a few borderline images.

2c. Test set

The test images were not used until this step.

In [8]:
```
test_pred <- predict(final_fit, z_test)$Prediction
test_pred <- factor(levels(y_train)[test_pred], levels = levels(y_train))
cat("test misclassification rate =", round(mean(test_pred != y_test), 4),
    " (", sum(test_pred != y_test), "of", length(y_test), ")\n")
table(predicted = test_pred, true = y_test)
```

```
test misclassification rate = 0.0189  ( 6 of 317 )
         true
predicted   1   6   7
        1 101   0   0
        6   0 106   3
        7   3   0 104
```

The selected forest misclassifies 6 of the 317 test images, a test error of 1.89%, which is in line with the OOB estimates of 2.14% and 2.45%. The six mistakes are three 7s called 6 and three 1s called 7, and no 6 is missed. A 1 with a serif or a slant shares strokes with a 7, which makes that pair the plausible confusion, while the closed loop of a 6 sets it apart from both.

Question 3: Planning a random-forest analysis skill

3a.

I would write the skill around the decisions I actually care about and leave the standard steps to a short checklist. A current model like Claude already knows how to fit a random forest, read OOB error and compute permutation importance, so long step-by-step instructions mostly repeat what it would do anyway. In my Homework 7 and 9 skills, the extra text even caused trouble where two rules quietly disagreed (a tie rule against an "admit fold noise" rule, a 20-setting cap against "widen at an edge"). What the AI cannot know is my choices, such as the selection rule, how the grid is built, which numbers must appear in which table, and what I do not want in the report. My plan is a skill of about 30 lines with three parts. The first is a few lines of non-negotiable basics that keep the analysis honest. The second is a numbered list of my preferences with the reason for each, and the third is a check the AI must run before it hands over the report. For a weaker model I would expand only the first part, because those are the steps it might skip, and keep my preferences as they are.

3b.

1. Fit an ordinary random forest with the `RLT` package (reinforcement off) and print the package version, the number of trees and the resampling scheme at the top of the

report.

2. Set one random seed at the start, derive every other seed from it, and state it in the report so the run reproduces.
3. Keep the test set untouched until the final model is chosen, and tune with OOB error (or cross-validation if there is no OOB estimate).
4. Fit any preprocessing such as PCA or scaling on the training data only and apply the same transformation to the test data.
5. Tune `mtry` and node size on a grid of at most about 40 settings, and widen the grid once if the best value sits on an edge.
6. Report the full grid of OOB errors as one `mtry` × node-size table, not only the winner.
7. Select with a one-standard-error style rule, preferring the larger node size and then the smaller `mtry` when settings are within noise, and say explicitly when the top settings differ by only one or two observations.
8. Show permutation importance as one horizontal bar chart sorted from largest to smallest, and warn that correlated predictors split their importance.
9. Make every figure readable on a printed page, using the Okabe-Ito colors, a different marker per series, axis labels with units and ticks at the grid values.
10. Keep the report to one page of prose with concrete numbers in the first sentence of each paragraph, interpret the tables instead of repeating them, and end with one sentence on what I would try next.